Lab Setup Guide: SmartLock Engagement

This guide builds the isolated lab for Assignment 2. Everything stays on a virtual network that cannot reach the outside, so the interception and replay work is safe and authorised.

Why separate machines

A network attacker sits between two hosts. Traffic between a client and server on the **same** machine travels over loopback and never reaches the wire, so it cannot be captured from another machine or intercepted with ARP spoofing. To make the on-path attacks real, the lock and the hub must be on **different VMs on a shared, isolated network**.

```
two VMs (a relay on Kali models the on-path attacker):

  VM2  Kali  10.10.10.12                      VM1  Lock  10.10.10.11
  +-----------------------------+             +------------------------+
  |  hub_client                 |   isolated  |                        |
  |  relay (optional, on-path)  |============>|  lock_server  :PORT    |
  |  tcpdump / Wireshark        |  host-only  |                        |
  +-----------------------------+             +------------------------+

three VMs (a real ARP-spoof man in the middle):

  VM2 Hub .12            VM3 Kali .13                  VM1 Lock .11
  hub_client  <------->  ettercap / bettercap  <------->  lock_server
                         (ARP spoof + sniff)
```

Step 1: Install while the VMs have internet

An isolated network cannot download anything, so do this first. Give each VM a **NAT** adapter, then on the VMs that run the lock or the hub:

```
sudo apt update
sudo apt install build-essential libssl-dev openssh-client openssh-server
```

On Kali, also install the tools you plan to use (for example `wireshark`, `tcpdump`, `hashcat`, `ettercap-text-only`, `bettercap`, and `mitmproxy`). Copy the assignment folder onto each VM, then build it:

```
make -C smartlock
```

This produces `lock_server`, `hub_client`, and `instance_generator`. On the **lock VM only**, create your instance once from your roll number:

```
cd smartlock
./instance_generator 23i-0001      # use your own roll; writes instance.txt
```

Do not run the generator again on another VM. Each run creates a new device RSA key, and v2 refuses a lock whose key differs from the one in the hub's copy of `instance.txt`.

Step 2: Move to the isolated network

Power the VMs off. In VirtualBox, change each adapter from NAT to the **same Host-Only network** (or the same "Internal Network"), then start them. Internal Network has no DHCP, so use the static addresses below for either type. Find your interface name, written `IFACE` below, with `ip -br link` (often `eth0` on Kali and `enp0s3` on Ubuntu). Set the address again after each reboot.

VM	Address
Lock	`10.10.10.11`
Hub, or Kali when it also runs the hub (two VMs)	`10.10.10.12`
Kali attacker (three VMs)	`10.10.10.13`

```
sudo ip addr flush dev IFACE
sudo ip addr add 10.10.10.11/24 dev IFACE     # use this VM's address
sudo ip link set IFACE up
ping -c 2 10.10.10.12                         # another lab VM: must reply
ping -c 2 -W 2 8.8.8.8                        # the internet: must fail
```

Then copy your instance from the lock VM to the VM that runs the hub, so both share it. On the lock VM, start the SSH service; on the hub VM, fetch the file:

```
sudo systemctl start ssh                                    # on the lock VM
scp USER@10.10.10.11:PATH/smartlock/instance.txt smartlock/   # on the hub VM
```

The lock VM prints its port when the lock starts; it is written `PORT` below.

Option A: two VMs (minimum)

- **VM1 (SmartLock device),** `10.10.10.11`**:** `./lock_server instance.txt`
- **VM2 (Kali),** `10.10.10.12`**:** runs the hub (`./hub_client instance.txt 10.10.10.11`) and all attack tools. It holds `instance.txt` only because the hub needs it; your attacks must work from what you capture.

What this supports: passive capture of the hub-to-lock link, the keying discrete log, the nonce-reuse analysis, PIN cracking, and replay. For the on-path attacks (downgrade, impersonation), model the attacker with a **relay** on Kali: a program that accepts the hub's connection on a port you choose, opens its own connection to the lock, and forwards the traffic, editing the lines you target. The wire is line-based `key:value` text (TAB separated), not JSON, so a relay edits whole lines.

Write a short C++ relay of your own, or use a tool that can rewrite traffic. For example, this mitmproxy command only forwards until you add your own addon script, which then rewrites the `HELLO` and `CERT` lines:

```
mitmproxy --mode reverse:tcp://10.10.10.11:PORT \
  --listen-port 8000 -s your_addon.py
```

Start the lock, then the relay, then point the hub at the relay using the optional port argument:

```
./hub_client instance.txt 127.0.0.1 8000
```

The hub-to-relay traffic is on `lo` and the relay-to-lock traffic is on `IFACE`, so capture both to show the rewrite. This faithfully models an attacker on the path.

Option B: three VMs (full man in the middle)

- **VM1 (SmartLock),** `10.10.10.11`**:** `./lock_server instance.txt`
- **VM2 (Hub),** `10.10.10.12`**:** `./hub_client instance.txt 10.10.10.11`
- **VM3 (Kali attacker),** `10.10.10.13`**:** interception tools. It does not need `instance.txt`.

All three on the same network. From Kali, run a real ARP-spoofing man in the middle so the hub-to-lock traffic flows through Kali:

```
sudo sysctl -w net.ipv4.ip_forward=1
sudo ettercap -T -i IFACE -M arp /10.10.10.11// /10.10.10.12//
# or:
sudo bettercap -iface IFACE -eval \
"set arp.spoof.targets 10.10.10.11,10.10.10.12; arp.spoof on; net.sniff on"
```

Now Kali can read, modify, and drop the handshake and records live.

Capturing

- Wireshark on the lab interface (`IFACE`), or on `lo` if you keep the lock and hub on one VM for the capture-only tasks.
- Use "Follow TCP Stream" to read the newline-delimited handshake and records; each line is `key:value` fields separated by TABs.
- Save a `.pcap` as evidence for each finding.

```
sudo tcpdump -i IFACE -w capture.pcap tcp port PORT
```

Your first capture (do this before anything else)

1. On the lock VM, build, generate your instance, and start the lock. Note the port it prints.
2. On the capturing VM, start `tcpdump` (or Wireshark) on the lab interface, filtered to that port.
3. Run the hub once and let it finish.
4. Open the capture and use **Follow TCP Stream**. Confirm you can read every message type in order: `HELLO` , `SUITE` , `CERT` , `PAIR` , `DH` , `DHCLIENT` , and the `MSG` records.
5. Save the `.pcap` . This single capture is the starting point for most of Part A; `FUNCTIONALITY.pdf` has the field-by-field table of what you should see.

Which tool for which stage

A reminder that the evidence for each stage comes from a tool plus your own reasoning, not from one scanner. The Appendix explains each attack in full.

Stage	What you do	Tool	Evidence to keep
Any (capture)	record one normal session	`tcpdump` / Wireshark	a `.pcap` you can Follow-TCP-Stream
Negotiation	rewrite `HELLO` to `NULL` on the path	a relay of your own, or `mitmproxy` + addon, or `ettercap`	a capture showing `suites:NULL` and a cleartext greeting

Stage	What you do	Tool	Evidence to keep
Authentication (impersonation)	present your own `CERT` and `DH`, relay both sides	a relay, or `ettercap` / `bettercap`	two separate key exchanges, and records you can read
Authentication (PIN)	crack the captured MD5	`hashcat` / John the Ripper	the recovered PIN
Keying	discrete log of `B`, rebuild the key, decrypt the greeting	your own C++ (baby-step giant-step) + `openssl`	the recovered `b` and the flag
Message protection	XOR two records that share the nonce	your own C++ / CyberChef	`P1 xor P2` and one recovered plaintext
Teardown	resend a captured `UNLOCK` on a new connection	your own socket client	the unlock counter rising

Replay

- Open a **new TCP connection** to the lock and resend the recorded lines from your capture with a short socket program. `tcpreplay` alone is not enough: it resends the old packets with their old TCP sequence numbers, which do not belong to any new connection on the lock.
- Show the unlock counter increasing on each replay. It lasts across sessions, so it keeps climbing until you restart `lock_server`. To read it, run the hub once after your replays: its first `STATUS` reply shows the current `unlocks=` value.

PIN cracking

```
# the pairing PIN is a dictionary word plus three digits; a wordlist is provided
hashcat -m 0 -a 6 captured.md5 ../smartlock/pin_wordlist.txt ?d?d?d
```

Here `captured.md5` is the single `pinhash` value you read from the `PAIR` line. `-a 6` appends the three-digit mask to each word, so the search is a few thousand candidates and finishes at once.

Safety and scope

- Keep the network Host-Only or Internal. Do not bridge to the host network or the internet.
- Run ARP spoofing only between your own two VMs.
- Include a short "Scope and ethics" paragraph in your report confirming all work stayed inside this lab.